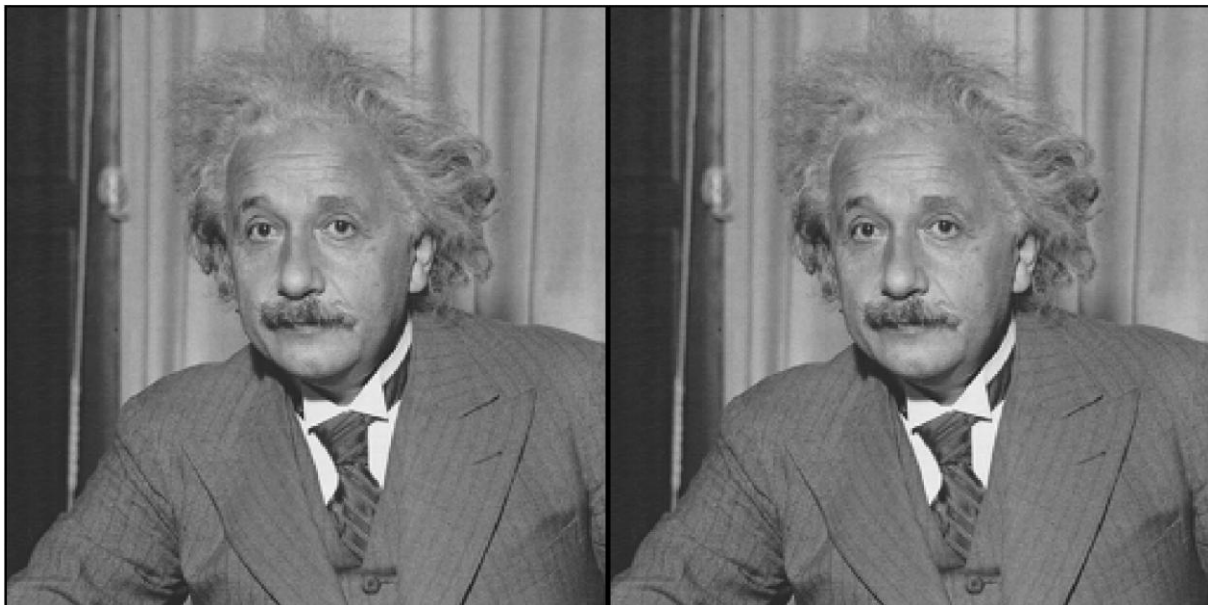


4TN4 Quiz 2

1. The below images show the original cameraman image and the embedded, binarized version of the cameraman image that was inserted that the desired bits (LSB, 1, and 2).

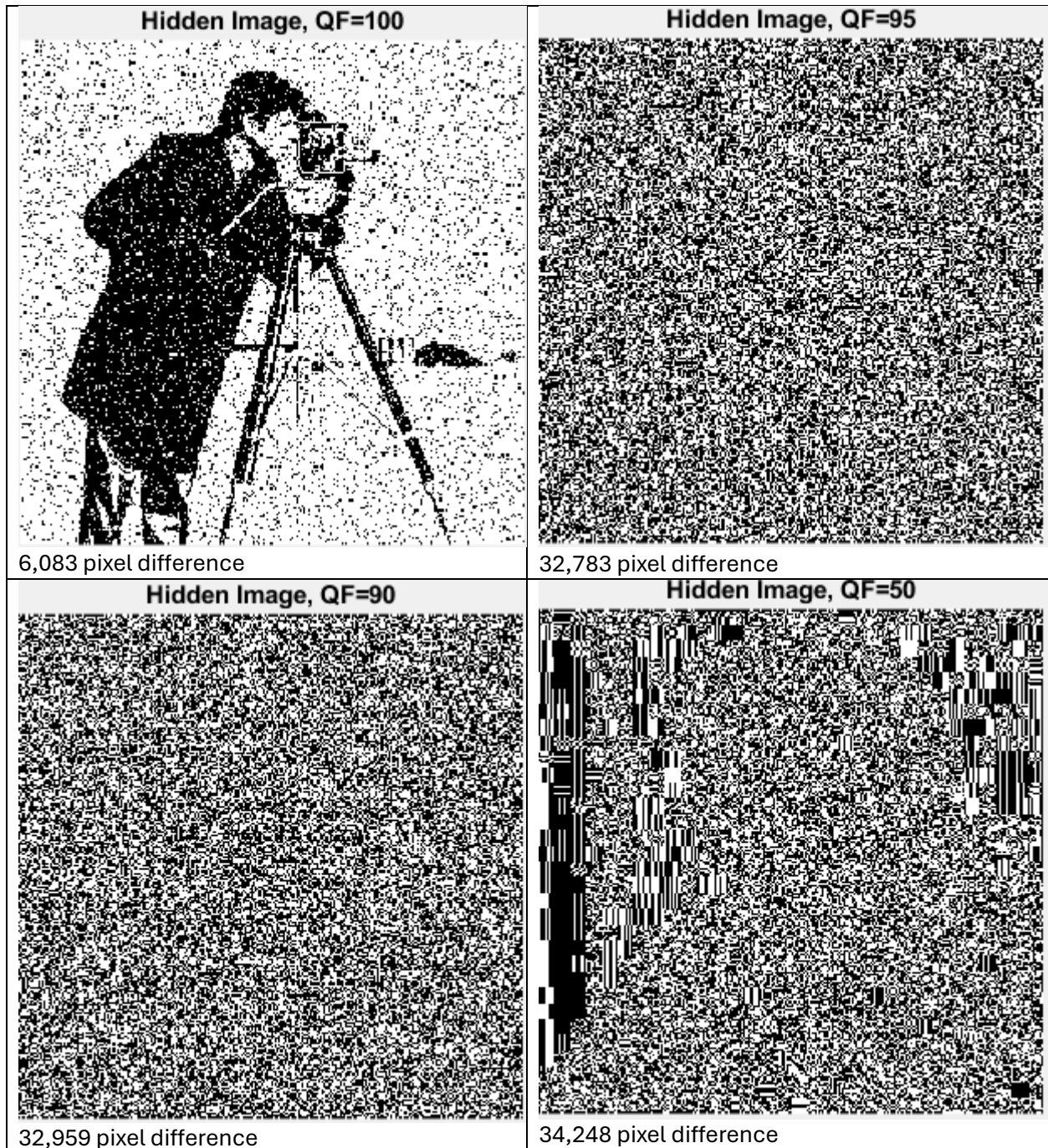


After doing the initial hiding, there is no visible differences between the images. The image on the left is the original “einstein.tif”, while the image on the right contains the binarized version of “cameraman.tif” as it’s LSB plane.



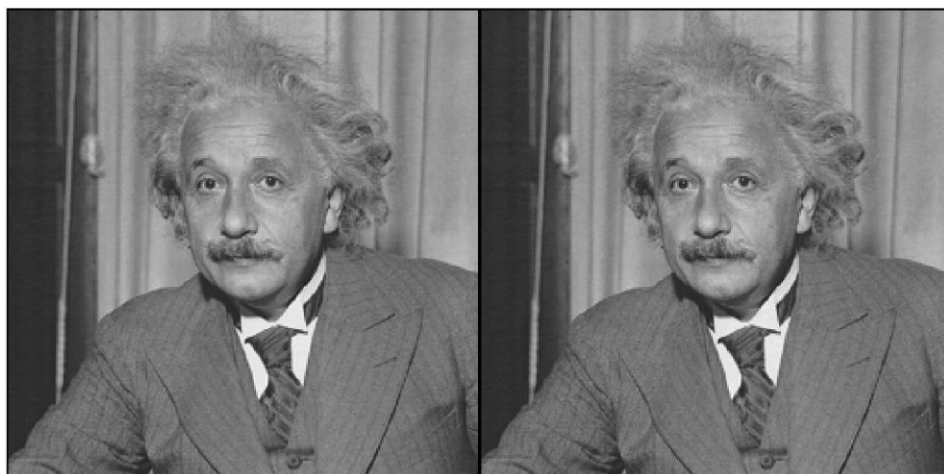
After compressing and decompressing using a factor of 100, 95, 90, and 50, it is obvious that this method is NOT very robust against a compression attack. Even at 100% quality factor, there is noticeable difference in the image, although it can still be identified. However, as soon as there is

any reduction in the quality factor the image becomes unrecoverable. As the quality factor decreases, the recoverability of the hidden image drastically decreases.



The results of the 1-bit plane being changed:

Again, there is no noticeable difference between the original einstein to the left and the modified to the right.

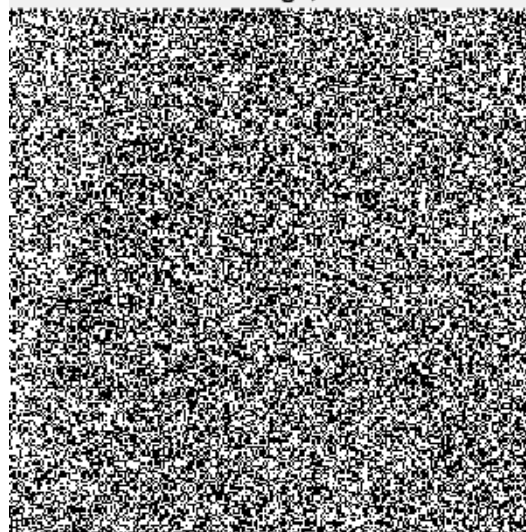


Hidden Image, QF=100



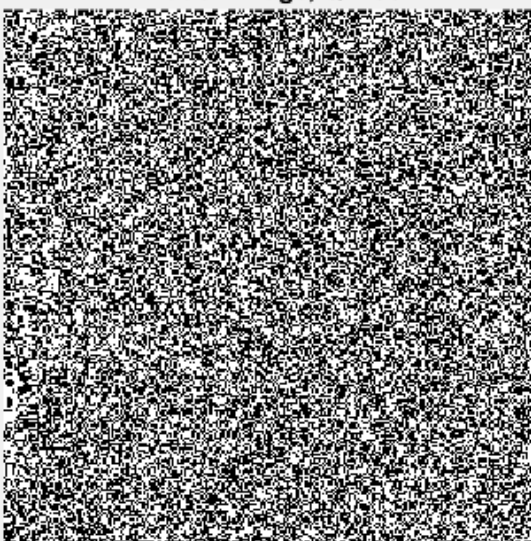
3,142 pixel difference

Hidden Image, QF=95



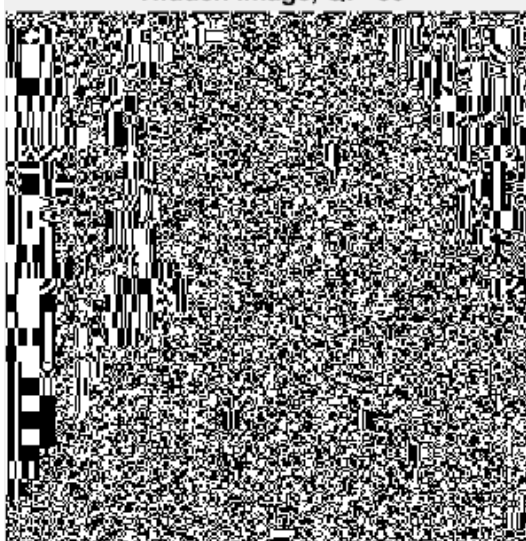
31,459 pixel difference

Hidden Image, QF=90



32,586 pixel difference

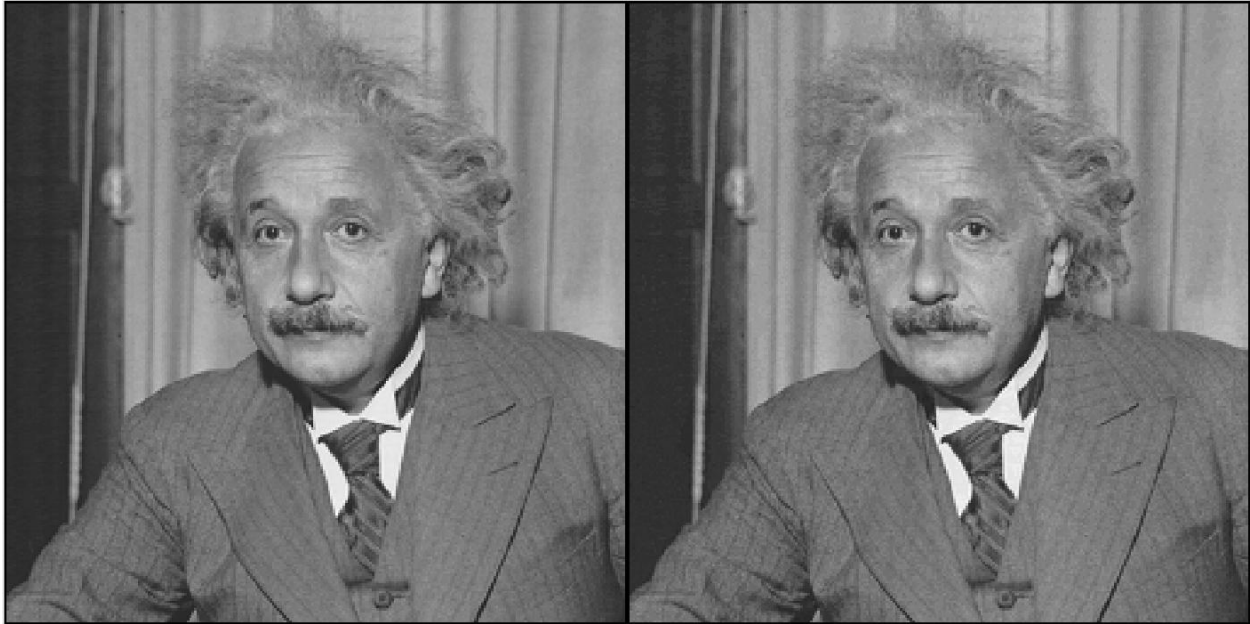
Hidden Image, QF=50



33,152 pixel difference

While there is a noticeable improvement in the number of pixels that are different, especially at quality factor 100, it is still extremely susceptible to compression attacks. Any quality factor below 100 has roughly half of the pixels different, so there is no useful information to be drawn.

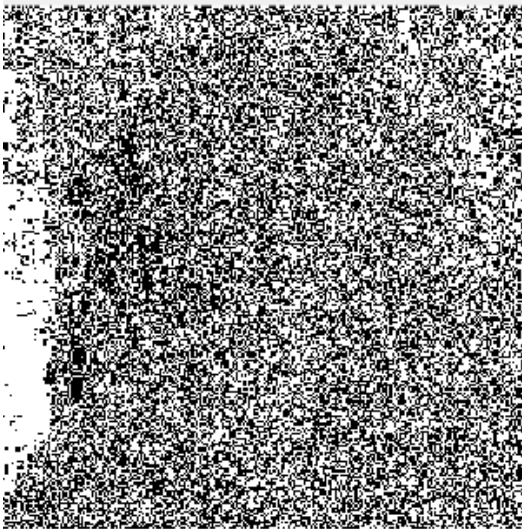
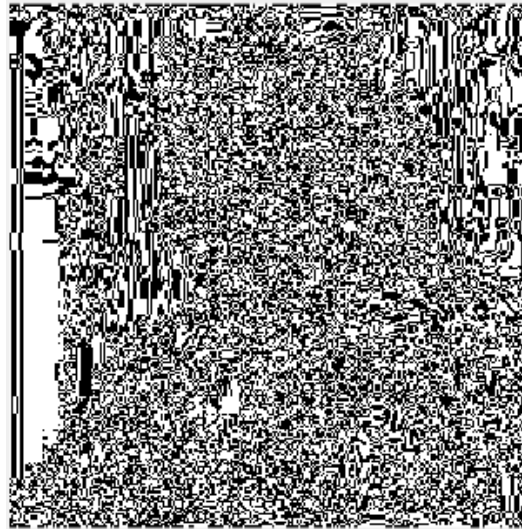
The results of the 2-bit plane being changed:



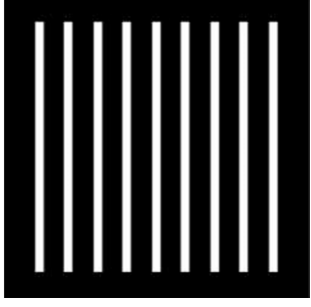
There could be a case made that there is very slight distortion of the original image, specifically an additional amount of contouring around the left eye (as that is where the binarized cameraman image is all black). However, this is very hard to see and would not be noticeable if I was not explicitly looking for it.

This can be further corroborated by the fact that now at the 95% quality factor, there is still a quite obvious mirage of the original binarized file. Even the 90% quality factor gives some indication of the original image, specifically in the lefthand corner. However, this is not significant enough to determine what the original image was.

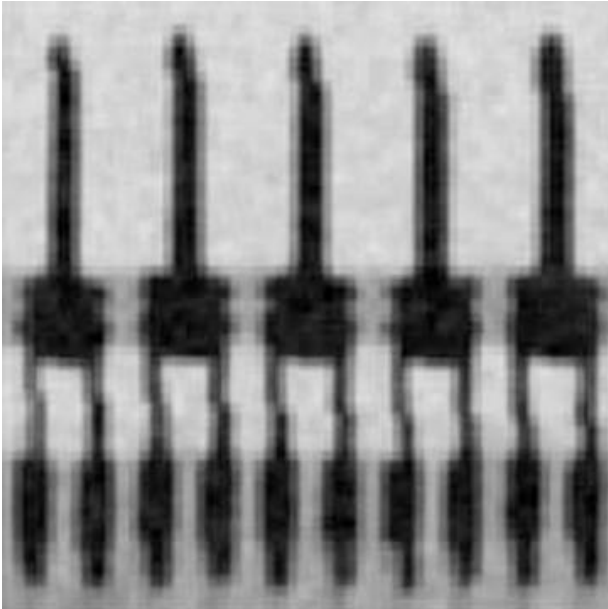
While it is quite faint, it seems that if the bit plane chosen is high enough the system is SOMEWHAT resistant to light compression. But generally, this method of data hiding cannot deal with compression.

Hidden Image, QF=100  1,633 pixel difference	Hidden Image, QF=95  22,076 pixel difference
Hidden Image, QF=90  28,482 pixel difference	Hidden Image, QF=50  31,280 pixel difference

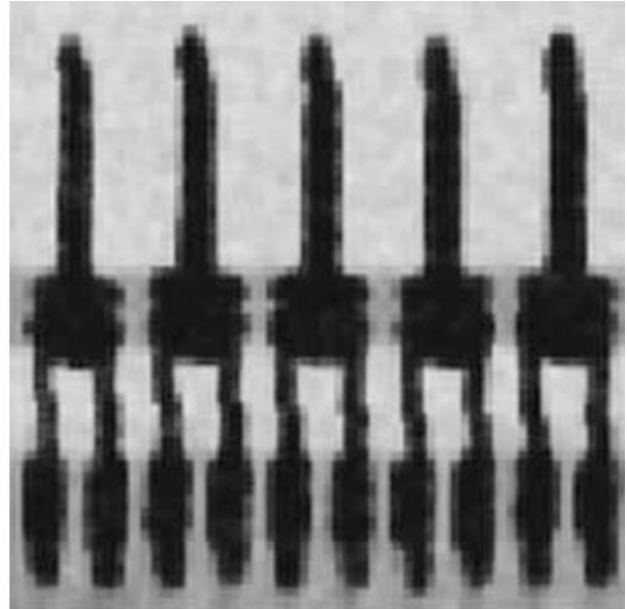
2. Arithmetic mean filtering works by taking replacing each pixel's value with the average of its neighbouring pixels. This effectively means a different amount of smoothing of edges depending on the size of the filter. When applied to the following image:

	a) 3x3 filter will provide very light smoothing of the edges of the white bars, with a hardly visible amount of smoothing
	b) 7x7 filter will provide much more smoothing, with the space between the bars being noticeably smaller (as there should be some amount of grey 7 pixels from each edge)
	c) 9x9 filter will provide even more smoothing, with the ends rounded and even less space between bars. There should now be no true black pixels between any of the lines, although it should be hardly noticeable.

3. 3x3 Arithmetic Mean



3x3 Geometric Mean



- a) The subimage obtained with geometric mean is less blurry due to the way that geometric mean filtering works. Geometric mean filtering takes the product of all the pixel, to the $1/(n \times m)$ power. So in this case, it would be the 3×3 area to the $1/9^{\text{th}}$ power. This means that if any pixel value is 0 (which would be the case of any black pixels), the product would also be 0. This leads to much sharper lines.

On the other hand, the arithmetic mean takes the average of the pixels in the kernel (the 3×3 square), meaning that there will smoothing of edges. This leads to more blurring as compared to the geometric mean filtering.

- b) Due to the geometric mean using the product of the surrounding pixels, the filter is more sensitive to lower intensity values and will make the pixels near the black regions also be become black. The arithmetic mean will create a gradient, where there will be higher intensity values as we move further from the black regions. However, these gradient areas will be just pure black in the geometric mean filter.

4.

```

21  21  21  95  169  243  243  243
21  21  21  95  169  243  243  243
21  21  21  95  169  243  243  243
21  21  21  95  169  243  243  243

```

- a) Entropy is calculated with $H = -\sum_{j=1}^J P(a_j) \log_2 P(a_j)$. $P(21) = 12/32$, $P(95) = 4/32$, $P(169) = 4/32$, $P(243) = 12/32$. $\therefore H = -\left(\frac{12}{32} \log_2 \frac{12}{32} + \frac{4}{32} \log_2 \frac{4}{32} + \frac{4}{32} \log_2 \frac{4}{32} + \frac{12}{32} \log_2 \frac{12}{32}\right) = 1.81 \text{ bits/pixel}$.

- b) After performing Huffman coding we have the following

21	0.375	0.375	0.625	0
243	0.375	0.375	0.375	1
95	0.125	0.25		
169	0.125			

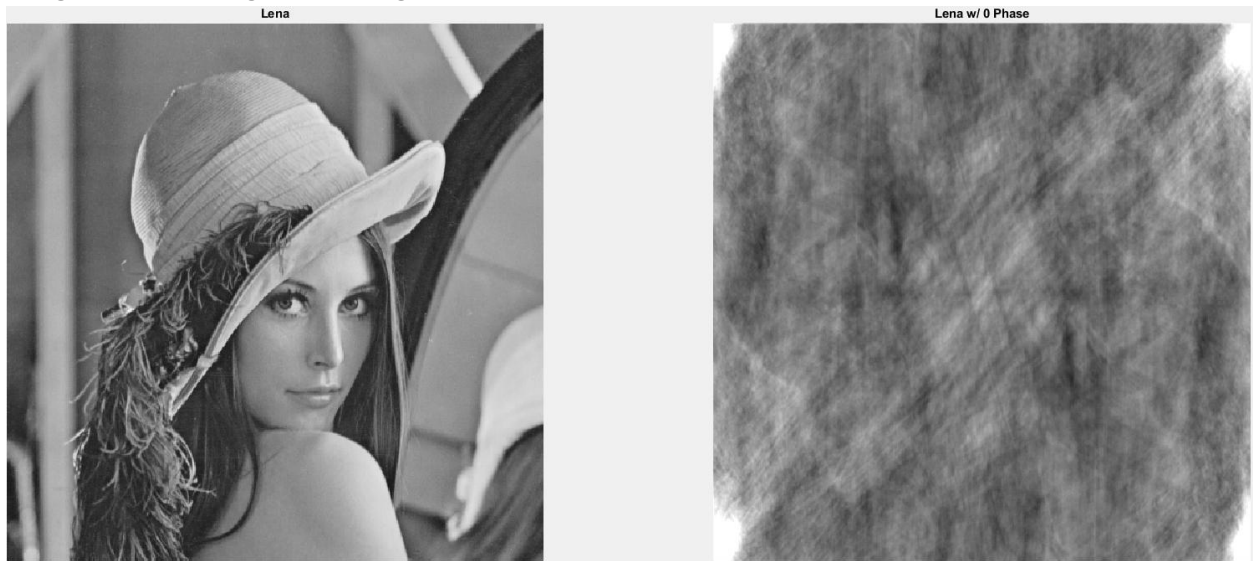
21	0.375	1	0.375	1	0.625	0
243	0.375	00	0.375	00	0.375	1
95	0.125	010	0.25	01		
169	0.125	011				

Which becomes the compressed image:

1	1	1	010	011	00	00	00
1	1	1	010	011	00	00	00
1	1	1	010	011	00	00	00
1	1	1	010	011	00	00	00

- c) Original image size: 32 pixels x 8 bits = 256 bits. Compressed size = 32 pixels x 3 bits = 96 bits. $\therefore$ Compression Ratio = $256/96 = 2.67$. $L_{avg} = (0.375)(1) + (0.375)(2) + (0.125)(3) + (0.125)(3) = 1.875 \text{ bits/pixel}$. This means that Huffman coding is very effective for this task, with only a 0.065 bit/pixel difference (aka loss of efficiency).
- d) When looking at pairs of pixels, we now have the "a" values of (21,21), (21,95), (95, 169), (169, 243), (243,243), and (243, 21). From the above image, we have: $P((21,21)) = 8/32$, $P((21,95)) = 4/32$, $P((95,169)) = 4/32$, $P((169,243)) = 4/32$, $P((243,243)) = 8/32$, and $P((243,21)) = 4/32$. $H = -\frac{1}{2} (2 * \frac{8}{32} \log_2 \frac{8}{32} + 4 * \frac{4}{32} \log_2 \frac{4}{32}) = \frac{1}{2} * 2.5 = 1.25 \text{ bits/pixel}$. Since the pixels are in pairs, the calculated entropy has to be divided by 2 to find the entropy per pixel instead of per pixel pair.
- e) The coded difference between adjacent pixels is as follows:
- | | | | | | | |
|---|---|----|----|----|---|---|
| 0 | 0 | 74 | 74 | 74 | 0 | 0 |
| 0 | 0 | 74 | 74 | 74 | 0 | 0 |
| 0 | 0 | 74 | 74 | 74 | 0 | 0 |
| 0 | 0 | 74 | 74 | 74 | 0 | 0 |
- This means that there is $P(0) = 16/28$ and $P(74) = 12/28$. This means the entropy is $H = -(\frac{16}{28} \log_2 \frac{16}{28} + \frac{12}{28} \log_2 \frac{12}{28}) = 0.98 \text{ bits/pixel}$. This tells us that we can compress the image even more than we already have, and it will lead to using much few bits (roughly half as many as when using the entropy of the original image).
- f) The entropy in a) is the default entropy of the image, without compression. This just directly reflects the probabilities of each symbol. The entropy in b) is lower, as there is now correlation between adjacent pixels which means that the probability of for example (21,21) occurring is higher than in the original method (where you would need to take $P(21)*P(21)$). The entropy in c) is lower still since most of the differences are 0 and the remaining differences are identical, meaning there are fewer different probabilities (which increases entropy), leading to the lowest entropy of all the methods tested.

5. After taking the FFT of lena.tif, removing the phase, and converting back using the IFFT, the image looks nothing like the original.



This is because phase effectively determines “where” the features of an image will be, like edges and contours. All that the magnitude of the image tells us the intensities of the frequencies, and without a phase associated with the magnitudes the image becomes nonsensical.

On the Elaine image:

